



Software Factory en la Era de la IA

Calidad Seguridad

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Qa Moderno

QA Moderno: Más Allá del Testing Manual

Evolución del QA

El QA tradicional centrado en testing manual está siendo reemplazado por un enfoque de **Quality Engineering** donde todos los roles contribuyen a la calidad.

Era	Enfoque	Rol QA	Limitación
2000s	Testing manual	Executor de tests	Bottleneck
2010s	Automation	Configurador de herramientas	Silo separado
2020s	Quality Engineering	Strategist de calidad	—

Principles del QA Moderno

1. Shift-Left

Mover la detección de defectos hacia etapas tempranas del ciclo de vida.

Traditional: Code → Test → Fix → Test again
Shift-Left: Design → Review → Code → Test (early) → Deploy

2. Automation-First

La automatización es la estrategia por defecto, manual es la excepción.

3. Quality as Code

Las reglas de calidad se definen en código, no en documentos.

```
# Ejemplo: Quality Gates como código
quality_gates:
  - name: "coverage"
    threshold: 80
    metric: "line_coverage"
  - name: "complexity"
    threshold: 10
    metric: "cyclomatic_complexity"
```

Roles en QA Moderno

Rol	Responsabilidad	Herramientas
SDET	Desarrollo de testing	Selenium, Cypress
Quality Coach	Estrategia y mentoring	Procesos, métricas
Platform Engineer	Infraestructura de testing	Docker, K8s
Security Champion	Seguridad en calidad	SAST, DAST

Métricas de Calidad

Métrica	Objetivo	Medición
Defect Escape Rate	< 5%	Bugs en producción
Test Coverage	> 80%	Cobertura de código
Mean Time to Detect	< 1 hora	Tiempo para detectar issues
Automated Test Ratio	> 70%	Tests automatizados vs total

Implementación

Fase 1: Assessment

Evaluar el estado actual de QA en la organización.

Fase 2: Strategy

Definir estrategia de testing basada en riesgos.

Fase 3: Tooling

Seleccionar e integrar herramientas en CI/CD.

Fase 4: Culture

Fomentar cultura de calidad compartida.

Ver también

- Pirámide de Testing
- Testing Automatizado
- Quality Gates
- DORA Metrics

Testing Piramide

La Pirámide de Testing

Definición

La pirámide de testing es un modelo que define la proporción ideal de diferentes tipos de tests en un proyecto de software.

Estructura de la Pirámide



E2E Tests (pocos, lentos, caros)

Integration Tests (moderados)

Unit Tests (muchos, rápidos, baratos)

Tipos de Tests

Unit Tests (Base)

- **Proporción:** 70% del total
- **Velocidad:** Milisegundos
- **Costo:** Bajo
- **Alcance:** Funciones, métodos aislados

```
# Ejemplo: Unit test con pytest
def test_calculate_discount():
    product = Product(price=100)
    assert product.apply_discount(10) == 90

def test_calculate_discount_invalid():
    product = Product(price=100)
    with pytest.raises(ValueError):
        product.apply_discount(150)
```

Integration Tests (Medio)

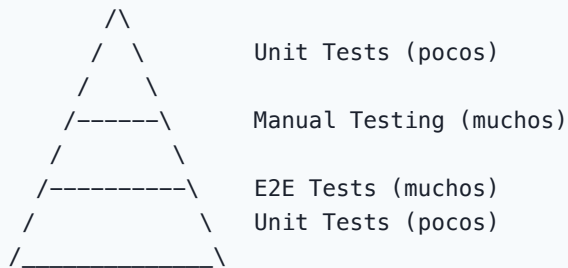
- **Proporción:** 20% del total
- **Velocidad:** Segundos
- **Costo:** Medio
- **Alcance:** Interacción entre componentes

```
// Ejemplo: Integration test con Jest
describe('UserService', () => {
  it('should create user and send welcome email', async () => {
    const user = await userService.create({
      name: 'Test',
      email: 'test@example.com'
    });
    expect(user.id).toBeDefined();
    expect(emailService.sendWelcome).toHaveBeenCalledWith(user);
  });
});
```

E2E Tests (Punta)

- **Proporción:** 10% del total
- **Velocidad:** Minutos
- **Costo:** Alto
- **Alcance:** Flujos completos de usuario

Ice Cream Cone Anti-Pattern



Señales de alerta:

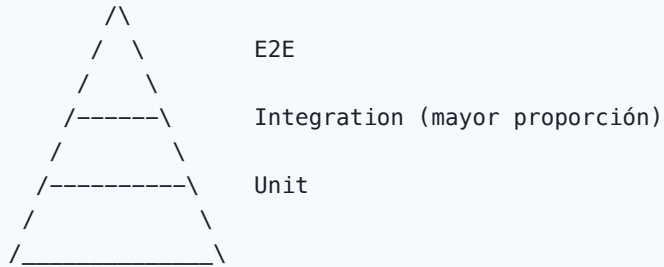
- Más tests manuales que automatizados
- Tests E2E lentos y frágiles
- Poca cobertura de unit tests
- Builds que tardan horas

Proporciones Recomendadas

Tipo	Proporción	Velocidad	ROI
Unit	70%	< 100ms	Alto
Integration	20%	< 5s	Medio
E2E	10%	< 60s	Bajo

Adaptaciones Modernas

Testing Trophy (Kent C. Dodds)



Testing Diamond

Prioriza integration testing sobre unit testing.

Ver también

- QA Moderno
- Testing Automatizado
- Performance Testing
- Quality Gates
- Prevención de Defectos

Automated Testing

Testing Automatizado

Frameworks por Lenguaje

JavaScript/TypeScript

Framework	Tipo	Ventajas	Uso
Jest	Unit/Integration	Zero config, snapshots	React, Node.js
Vitest	Unit/Integration	ESM, rápido	Vite projects
Cypress	E2E	Debugging, DX	Frontend
Playwright	E2E	Multi-browser, parallel	Full-stack

```
// Jest: Unit test
describe('Calculator', () => {
  it('adds numbers correctly', () => {
    expect(add(2, 3)).toBe(5);
  });
});

// Playwright: E2E test
test('user can login', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[data-testid="email"]', 'user@test.com');
  await page.fill('[data-testid="password"]', 'pass123');
  await page.click('[data-testid="submit"]');
  await expect(page).toHaveURL('/dashboard');
});
```

Python

Framework	Tipo	Ventajas	Uso
pytest	Unit/Integration	Fixtures, plugins	General
Selenium	E2E	Multi-language	Web apps
Playwright	E2E	Async, modern	Web apps

```
# pytest: Unit test
def test_user_creation():
```

```

    user = User(name="Test", email="test@test.com")
    assert user.is_valid() == True

# pytest: Integration test with fixtures
@pytest.fixture
def database():
    db = create_test_database()
    yield db
    db.cleanup()

def test_insert_user(database):
    user = User(name="Test")
    database.insert(user)
    assert database.get(user.id) is not None

```

Java

Framework	Tipo	Ventajas	Uso
JUnit 5	Unit/Integration	Standard, extensions	Enterprise
Mockito	Unit (mocks)	Mocking framework	Any Java
Selenium	E2E	Multi-browser	Web apps
RestAssured	Integration	API testing	REST APIs

```

// JUnit 5: Unit test
@Test
void shouldCalculateTotal() {
    Order order = new Order();
    order.addItem(new Item("Widget", 10.0));
    assertEquals(10.0, order.getTotal(), 0.001);
}

// RestAssured: Integration test
@Test
void shouldReturnUser() {
    given()
        .contentType(ContentType.JSON)
    .when()
        .get("/api/users/1")
    .then()
        .statusCode(200)
        .body("name", equalTo("John"));
}

```

Estrategias de Testing

Test Data Management

```
# factory_bot.yml
user:
  name: "Test User"
  email: "test@example.com"
  role: "admin"

product:
  name: "Test Product"
  price: 29.99
  sku: "TEST-001"
```

Mocking y Stubbing

```
// Jest: Mocking
jest.mock('./emailService');
emailService.send.mockResolvedValue({ success: true });

// Spy
const spy = jest.spyOn(logger, 'error');
await service.process();
expect(spy).toHaveBeenCalledWith('Error occurred');
```

Parallel Execution

```
# playwright.config.ts
export default {
  workers: 4, // Parallel tests
  retries: 2, // Retry flaky tests
  use: {
    baseURL: 'http://localhost:3000'
  }
};
```

CI/CD Integration

```
# GitHub Actions example
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
    - run: npm ci
    - run: npm run test:unit
    - run: npm run test:integration
```

```
- run: npm run test:e2e
- uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: test-results/
```

Métricas de Automatización

Métrica	Target	Descripción
Code Coverage	> 80%	Líneas ejecutadas
Test Pass Rate	> 95%	Tests que pasan
Mean Test Time	< 5min	Tiempo promedio
Flakiness Rate	< 2%	Tests inestables

Ver también

- Pirámide de Testing
- Performance Testing
- Security Testing
- Quality Gates
- Pipelines CI/CD

Performance Testing

Performance Testing y Load Testing

Tipos de Performance Testing

Tipo	Objetivo	Preguntas que responde
Load Testing	Carga esperada	¿Funciona bajo carga normal?
Stress Testing	Más allá del límite	¿Qué pasa cuando se satura?
Spike Testing	Picos repentinos	¿Soporta tráfico viral?
Endurance Testing	Larga duración	¿Hay memory leaks?
Scalability Testing	Escalabilidad	¿Crece con la demanda?

Herramientas Comparación

Herramienta	Lenguaje	Tipo	Curva aprendizaje
k6	JavaScript	Script	Baja
Gatling	Scala/Java	Script	Media
JMeter	Java	GUI/XML	Alta
Locust	Python	Script	Baja
Artillery	YAML/JS	Config	Baja

k6 (Recomendado)

Instalación

```
# macOS
brew install k6

# Docker
docker pull grafana/k6
```

Script Básico

```
import http from 'k6/http';
import { check, sleep } from 'k6';
```

```

export const options = {
  stages: [
    { duration: '30s', target: 20 }, // Ramp up
    { duration: '1m', target: 20 }, // Stay at 20 users
    { duration: '30s', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% under 500ms
    http_req_failed: ['rate<0.01'], // < 1% errors
  },
};

export default function () {
  const res = http.get('https://api.example.com/users');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });
  sleep(1);
}

```

Escenarios Avanzados

```

export const options = {
  scenarios: {
    constant_load: {
      executor: 'constant-vus',
      vus: 50,
      duration: '5m',
    },
    spike_test: {
      executor: 'ramping-vus',
      stages: [
        { duration: '1m', target: 10 },
        { duration: '10s', target: 200 }, // Spike
        { duration: '1m', target: 10 },
      ],
    },
  },
};

```

Locust

```

from locust import HttpUser, task, between

class WebsiteUser(HttpUser):
    wait_time = between(1, 3)

```

```
@task(3) # Weight: 3x more likely
def view_products(self):
    self.client.get("/products")

@task(1)
def purchase(self):
    self.client.post("/orders", json={
        "product_id": 1,
        "quantity": 2
    })
```

JMeter

```
<?xml version="1.0" encoding="UTF-8"?>
<jmeterTestPlan>
  <TestPlan>
    <ThreadGroup numThreads="100" rampTime="30">
      <HTTPSampler domain="api.example.com"
        path="/users"
        method="GET"/>
      <ResponseAssertion>
        <testStrings>
          <string>200</string>
        </testStrings>
      </ResponseAssertion>
    </ThreadGroup>
  </TestPlan>
</jmeterTestPlan>
```

Métricas Clave

Métrica	Descripción	Target
Response Time	Tiempo de respuesta promedio	< 200ms
p95/p99	Percentiles de latencia	< 500ms / < 1s
Throughput	Requests por segundo	Según capacidad
Error Rate	Porcentaje de errores	< 0.1%
Apdex Score	Satisfacción del usuario	> 0.95

Thresholds y Alerts

```
// k6: Thresholds
export const options = {
  thresholds: {
```

```
'http_req_duration': [  
  'p(50)<200',    // Mediana < 200ms  
  'p(95)<500',    // 95th percentile < 500ms  
  'p(99)<1000',   // 99th percentile < 1s  
],  
'http_req_failed': ['rate<0.01'],  
'http_reqs': ['rate>100'], // > 100 req/s  
},  
};
```

CI/CD Integration

```
# GitHub Actions con k6  
performance:  
  runs-on: ubuntu-latest  
  steps:  
    - uses: actions/checkout@v4  
    - name: Run k6 tests  
      uses: grafana/k6-action@v0.3.1  
      with:  
        filename: tests/performance.js  
    - name: Upload results  
      uses: actions/upload-artifact@v4  
      with:  
        name: k6-results  
        path: results/
```

Ver también

- Testing Automatizado
- Monitoring y Observabilidad
- DORA Metrics
- Infrastructure as Code

Security Testing

Security Testing

Tipos de Security Testing

Tipo	Nombre Completo	Fase	Velocidad	Cobertura
SAST	Static Application Security Testing	Desarrollo	Rápida	Código
DAST	Dynamic Application Security Testing	Runtime	Lenta	Aplicación
SCA	Software Composition Analysis	Build	Media	Dependencias
IAST	Interactive AST	Runtime	Media	Código + Runtime
RASP	Runtime Application Self-Protection	Runtime	Lenta	Protección activa

SAST (Static Analysis)

Herramientas

Herramienta	Lenguaje	Open Source	Integración
SonarQube	Multi	Community	CI/CD
Semgrep	Multi	Sí	CI/CD, IDE
CodeQL	Multi	Sí	GitHub
Checkmarx	Multi	No	Enterprise

Semgrep

```
# .semgrep.yml
rules:
  - id: sql-injection
    patterns:
      - pattern: |
          $QUERY = "SELECT * FROM users WHERE id = " + $ID
        message: "Potential SQL injection"
        languages: [python]
        severity: ERROR

  - id: hardcoded-secret
    pattern: |
      password = "..."
```

```
message: "Hardcoded password detected"
languages: [python]
severity: WARNING
```

```
# Uso
semgrep --config=auto .
semgrep --config=p/owasp-top-ten .
```

SonarQube

```
# sonar-project.properties
sonar.projectKey=my-project
sonar.sources=src
sonar.host.url=http://localhost:9000
sonar.qualitygate.wait=true

# Quality Gate
sonar.qualitygate.conditions=coverage > 80,duplicated_lines < 3
```

DAST (Dynamic Analysis)

Herramientas

Herramienta	Tipo	Costo	Curva Aprendizaje
OWASP ZAP	Open Source	Gratis	Media
Burp Suite	Comercial	\$\$	Baja
Nikto	Open Source	Gratis	Alta
Nuclei	Open Source	Gratis	Media

OWASP ZAP

```
# Docker scan
docker run -t ghcr.io/zaproxy/zaproxy:stable \
  zap-full-scan.py \
  -t https://example.com \
  -r report.html

# API scan
docker run -t ghcr.io/zaproxy/zaproxy:stable \
  zap-api-scan.py \
  -t https://example.com/openapi.json \
```



```
-f openapi \  
-r api-report.html
```

ZAP en CI/CD

```
# GitHub Actions  
security-scan:  
  runs-on: ubuntu-latest  
  steps:  
    - name: ZAP Baseline Scan  
      uses: zaproxy/action-baseline@v0.12.0  
      with:  
        target: https://staging.example.com  
        rules_file_name: 'zap-rules.tsv'  
        cmd_options: '-a'
```

SCA (Software Composition Analysis)

Herramientas

Herramienta	Cobertura	Integración	Costo
Snyk	Dependencies	CI/CD, IDE	Freemium
Dependabot	GitHub deps	GitHub	Gratis
OWASP Dep-Check	CVEs	CI/CD	Gratis
Trivy	Containers	CI/CD	Gratis

Snyk

```
# Instalación  
npm install -g snyk  
  
# Escaneo  
snyk test  
snyk monitor  
  
# Container scan  
snyk container test myapp:latest
```

Dependabot

```
# .github/dependabot.yml  
version: 2  
updates:
```

```
- package-ecosystem: "npm"
  directory: "/"
  schedule:
    interval: "weekly"
  open-pull-requests-limit: 10
- package-ecosystem: "docker"
  directory: "/"
  schedule:
    interval: "weekly"
```

Integración en Pipeline

```
# Pipeline completo de security testing
stages:
  - name: SAST
    script:
      - semgrep --config=auto .
      - sonar-scanner
    allow_failure: false

  - name: SCA
    script:
      - snyk test
      - trivy fs .
    allow_failure: false

  - name: DAST
    script:
      - docker run -t zaproxy/zaproxy zap-full-scan.py -t $STAGING_URL
    allow_failure: true
    when: manual
```

Vulnerability Management

Severidad	SLA Fix	Ejemplo
Critical	24 horas	SQL Injection
High	1 semana	XSS
Medium	1 mes	Information Disclosure
Low	Next release	Minor issues

Ver también

- OWASP Top 10
- DevSecOps
- Compliance

- Zero Trust
- Supply Chain Security

Owasp Top10

OWASP Top 10

Visión General

El OWASP Top 10 es el estándar de referencia para las vulnerabilidades de seguridad más críticas en aplicaciones web.

Top 10 (2021)

Rank	Vulnerabilidad	Impacto
A01	Broken Access Control	Alto
A02	Cryptographic Failures	Alto
A03	Injection	Alto
A04	Insecure Design	Medio
A05	Security Misconfiguration	Medio
A06	Vulnerable Components	Alto
A07	Authentication Failures	Alto
A08	Data Integrity Failures	Medio
A09	Logging Failures	Medio
A10	SSRF	Alto

A01: Broken Access Control

Descripción: Los usuarios actúan fuera de sus permisos previstos.

```
// Vulnerable: No verifica permisos
app.get('/api/users/:id', (req, res) => {
  const user = db.users.findById(req.params.id); // Cualquier usuario puede acceder
  res.json(user);
});

// Seguro: Verifica permisos
app.get('/api/users/:id', authenticate, (req, res) => {
  if (req.user.id !== req.params.id && !req.user.isAdmin) {
    return res.status(403).json({ error: 'Forbidden' });
  }
  const user = db.users.findById(req.params.id);
```

```
res.json(user);
});
```

Prevención:

- Deny by default
- Re-usable access control modules
- Disable web server directory listing
- Log access control failures

A02: Cryptographic Failures

Descripción: Fallos en criptografía que exponen datos sensibles.

```
# Vulnerable: MD5 para passwords
import hashlib
def hash_password(password):
    return hashlib.md5(password.encode()).hexdigest()

# Seguro: bcrypt
import bcrypt
def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

Prevención:

- Clasificar datos procesados y almacenados
- No almacenar datos sensibles innecesariamente
- Usar algoritmos actualizados (AES-256, RSA)
- Usar Key Management properly

A03: Injection

Descripción: SQL, NoSQL, OS, LDAP injection.

```
# Vulnerable: SQL Injection
query = f"SELECT * FROM users WHERE id = {user_input}"

# Seguro: Parameterized query
query = "SELECT * FROM users WHERE id = %s"
cursor.execute(query, (user_input,))
```

```
// Vulnerable: XSS
element.innerHTML = userInput;

// Seguro: Text content
element.textContent = userInput;
```

```
// Or use DOMPurify for HTML
element.innerHTML = DOMPurify.sanitize(userInput);
```

A04: Insecure Design

Descripción: Fallos en el diseño de seguridad.

Patrón	Descripción
Threat Modeling	Identificar amenazas en diseño
Secure Design Patterns	Patrones probados
Reference Architecture	Arquitecturas seguras

A05: Security Misconfiguration

Descripción: Configuraciones de seguridad por defecto o incompletas.

```
# Vulnerable
server_tokens on; # Muestra versión de nginx

# Seguro
server_tokens off;
```

```
# Docker: Seguro
services:
  app:
    read_only: true
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
```

A06: Vulnerable and Outdated Components

Descripción: Usar componentes con vulnerabilidades conocidas.

```
# Detección
npm audit
snyk test
trivy fs .
```

Prevención:

- Mantener inventario de componentes (SBOM)
- Usar herramientas SCA
- Solo obtener componentes de fuentes oficiales
- Monitorear componentes sin mantenimiento

A07: Identification and Authentication Failures

Descripción: Fallos en autenticación y gestión de sesiones.

```
# Seguro: Rate limiting en login
from flask_limiter import Limiter

limiter = Limiter(app, default_limits=["5 per minute"])

@app.route('/login', methods=['POST'])
@limiter.limit("5 per minute")
def login():
    # Autenticación con MFA
    ...
```

A08: Software and Data Integrity Failures

Descripción: Fallos en integridad de software y datos.

- CI/CD pipeline integrity
- Signed commits
- Code signing
- Deserialization vulnerabilities

A09: Security Logging and Monitoring Failures

Descripción: Fallos en logging y monitoreo de seguridad.

```
# Logging de seguridad
import logging

security_logger = logging.getLogger('security')

def log_failed_login(username, ip):
    security_logger.warning(
        f"Failed login attempt for {username} from {ip}"
    )

def log_privilege_escalation(user, action):
    security_logger.critical(
        f"Privilege escalation: {user} attempted {action}"
    )
```

A10: Server-Side Request Forgery (SSRF)

Descripción: El servidor realiza requests no previstos.

```
# Vulnerable
import requests
def fetch_url(url):
```

```
        return requests.get(url).text # SSRF vulnerable

# Seguro
from urllib.parse import urlparse
import ipaddress

ALLOWED_HOSTS = ['api.example.com', 'cdn.example.com']

def fetch_url(url):
    parsed = urlparse(url)
    if parsed.hostname not in ALLOWED_HOSTS:
        raise ValueError("Host not allowed")
    return requests.get(url, timeout=5).text
```

Ver también

- Security Testing
- DevSecOps
- Compliance
- Zero Trust
- Supply Chain Security

Devsecops

DevSecOps: Seguridad Integrada

Definición

DevSecOps integra la seguridad en cada fase del ciclo de vida del desarrollo de software, automatizando las prácticas de seguridad.

```
Traditional DevOps:  
Code → Build → Test → Deploy → Monitor  
  
DevSecOps:  
Code → SAST → Build → SCA → Test → DAST → Deploy → RASP → Monitor
```

Principles

Principle	Descripción
Shift-Left	Seguridad temprana en el ciclo
Automation	Seguridad automatizada, no manual
Continuous	Seguridad continua, no puntual
Shared Responsibility	Seguridad es responsabilidad de todos

Pipeline de Seguridad

Etapas 1: Planificación

```
# Threat modeling en la planificación  
threat_model:  
  method: STRIDE  
  assets:  
    - user_data  
    - payment_info  
  threats:  
    - spoofing: MFA required  
    - tampering: Input validation
```

Etapas 2: Desarrollo

```
# Pre-commit hooks  
hooks:
```

```
pre-commit:
  - semgrep --config=auto
  - gitleaks detect
  - detect-secrets scan
```

Etapas 3: Build

```
# CI Pipeline Security
build:
  stages:
    - name: sast
      script: semgrep --config=p/owasp-top-ten .
    - name: sca
      script: snyk test --severity-threshold=high
    - name: container-scan
      script: trivy image myapp:${CI_COMMIT_SHA}
    - name: sbom
      script: syft scan dir:. -o spdx > sbom.json
```

Etapas 4: Deploy

```
# Deployment Security
deploy:
  pre_deployment:
    - dast_scan: zap-full-scan.py -t ${STAGING_URL}
    - policy_check: opa eval --data policy.rego --input deploy.json
  post_deployment:
    - smoke_test: security-check.sh
    - monitor_enable: falco --enabled
```

Herramientas por Fase

Fase	Herramienta	Tipo
IDE	SonarLint, Semgrep	SAST
Pre-commit	Gitleaks, detect-secrets	Secrets
CI/CD	Snyk, Trivy, SonarQube	SCA/SAST
Runtime	Falco, Sysdig	RASP
Monitoring	Splunk, ELK	SIEM

Secrets Management

Git Secrets

```
# Instalación
brew install git-secrets

# Uso
git secrets --install
git secrets --register-aws
git secrets --scan
```

HashiCorp Vault

```
# Vault: Obtener secrets
import hvac

client = hvac.Client(url='https://vault.example.com')
client.token = os.environ['VAULT_TOKEN']

# Leer secret
secret = client.secrets.kv.v2.read_secret_version(
    path='database/credentials'
)
db_password = secret['data']['data']['password']
```

GitHub Actions Secrets

```
# Uso de secrets en GitHub Actions
steps:
  - name: Deploy
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    run: aws s3 sync ./dist s3://my-bucket
```

Infrastructure Security

```
# Terraform: Security groups
resource "aws_security_group" "web" {
  name      = "web-sg"
  description = "Web server security group"

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # Only HTTPS
  }
}
```

```

egress {
  from_port    = 0
  to_port      = 0
  protocol     = "-1"
  cidr_blocks  = ["0.0.0.0/0"]
}
}

```

Container Security

```

# Dockerfile seguro
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM node:18-alpine
RUN addgroup -g 1001 -S appgroup && \
  adduser -S appuser -u 1001
USER appuser
COPY --from=builder --chown=appuser:appgroup /app/dist ./dist
EXPOSE 3000
CMD ["node", "dist/index.js"]

```

Kubernetes Security

```

# Pod Security
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1001
  containers:
    - name: app
      image: myapp:latest
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
      resources:
        limits:

```

memory: "256Mi"
cpu: "500m"

Métricas de DevSecOps

Métrica	Target	Descripción
Time to Remediate	< 24h (critical)	Tiempo para fix vulnerabilities
Security Tests Coverage	100%	Pipeline coverage
False Positive Rate	< 10%	Precisión de herramientas
Secrets Detected	0	Secrets en código

Ver también

- Security Testing
- OWASP Top 10
- Compliance
- Zero Trust
- Supply Chain Security
- Pipelines CI/CD

Compliance Regulaciones

Compliance y Regulaciones

Principales Frameworks

Framework	Alcance	Multas	Requisitos principales
GDPR	EU	Hasta 4% revenue	Protección datos personales
SOC 2	Global	Reputacional	Trust service criteria
HIPAA	USA (salud)	Hasta \$1.5M/año	Protected health info
PCI-DSS	Global (pagos)	\$5K-100K/mes	Card data security
ISO 27001	Global	—	ISMS framework

GDPR (General Data Protection Regulation)

Principios Clave

Principio	Descripción	Implementación
Lawfulness	Base legal para procesamiento	Consent, contract
Purpose Limitation	Solo para fines específicos	Data catalog
Data Minimization	Solo datos necesarios	Field validation
Accuracy	Datos precisos y actualizados	Update mechanisms
Storage Limitation	No más del necesario	Retention policies
Integrity	Seguridad adecuada	Encryption, access control
Accountability	Demostrar compliance	Documentation

Derechos del Usuario

```
# GDPR: Right to be forgotten
def delete_user_data(user_id):
    # Eliminar de todas las fuentes
    db.users.delete(user_id)
    cache.delete(f"user:{user_id}")
    search.index.delete(user_id)
    s3.delete(f"avatars/{user_id}")
```

```
# Registrar eliminación
audit_log.record("user_deleted", user_id)
return {"status": "deleted", "retention": "30_days_backup"}
```

Technical Implementation

```
# GDPR compliance checklist
gdpr:
  data_inventory:
    - user_email
    - user_name
    - ip_address
    - cookies
  consent:
    - cookie_consent_banner
    - privacy_policy
    - opt_in_required
  security:
    - encryption_at_rest: AES-256
    - encryption_in_transit: TLS 1.3
    - access_logging: enabled
    - data_masking: enabled
  rights:
    - data_export: api_endpoint
    - data_deletion: api_endpoint
    - consent_withdrawal: api_endpoint
```

SOC 2

Trust Service Criteria

Criterio	Descripción	Controles
Security	Protección contra acceso no autorizado	Firewalls, MFA
Availability	Operación del sistema	Uptime SLAs
Processing Integrity	Procesamiento correcto	QA, monitoring
Confidentiality	Restricción de datos	Encryption
Privacy	Uso de información personal	GDPR-like

Evidence Collection

```
# SOC 2 evidence automation
evidence:
  access_reviews:
```

```
frequency: quarterly
tool: okta
output: access-review-report.pdf
change_management:
  tool: jira
  queries:
    - "approved_changes_last_quarter"
    - "emergency_changes"
vulnerability_scans:
  tool: snyk
  frequency: weekly
  output: vulnerability-report.json
disaster_recovery:
  tool: aws-backup
  frequency: monthly
  output: dr-test-results.pdf
```

HIPAA

Protected Health Information (PHI)

Tipo	Ejemplo	Requisito
Identifiers	Nombre, SSN	Encryption
Health Data	Diagnósticos	Access control
Financial	Seguro médico	Audit logging
Technical	Device identifiers	De-identification

Implementation

```
# HIPAA: Audit logging
import logging
from datetime import datetime

hipaa_logger = logging.getLogger('hipaa_audit')

def access_phi(user_id, patient_id, action):
    hipaa_logger.info({
        'timestamp': datetime.utcnow().isoformat(),
        'user_id': user_id,
        'patient_id': patient_id,
        'action': action, # VIEW, CREATE, MODIFY, DELETE
        'ip_address': get_client_ip(),
        'justification': get_justification()
    })
```


PCI-DSS

Requirements

Requirement	Descripción
Network Security	Firewalls, segmentation
Data Protection	Encryption, masking
Vulnerability Management	Regular scans, patching
Access Control	Need-to-know, MFA
Monitoring	Logging, IDS
Policy	Security policies

Tokenization Example

```
# PCI: Tokenización de tarjetas
import stripe

def process_payment(card_number, amount):
    # Nunca almacenar el número completo
    token = stripe.Token.create(
        card={
            'number': card_number,
            'exp_month': exp_month,
            'exp_year': exp_year,
            'cvc': cvc,
        },
    )
    # Usar token, no el número real
    charge = stripe.Charge.create(
        amount=amount,
        currency='usd',
        source=token.id,
    )
    return charge.id
```

ISO 27001

ISMS Framework

```
Plan → Do → Check → Act

Plan:   Establish ISMS policy and objectives
Do:     Implement and operate ISMS
```

Check: Monitor and review ISMS
Act: Maintain and improve ISMS

Controls (Annex A)

Category	Controls
A.5	Information security policies
A.6	Organization of information security
A.7	Human resource security
A.8	Asset management
A.9	Access control
A.10	Cryptography
A.11	Physical security
A.12	Operations security
A.13	Communications security
A.14	System acquisition
A.15	Supplier relationships
A.16	Incident management
A.17	Business continuity
A.18	Compliance

Compliance Automation

```
# Open Policy Agent (OPA)
# policy.rego
package main

default allow = false

allow {
    input.method == "GET"
    input.path == ["api", "users"]
    input.user.role == "admin"
}

# Enforce encryption
allow {
```

```
    input.resource.encrypted == true  
}
```

Ver también

- DevSecOps
- Zero Trust
- Security Testing
- OWASP Top 10
- Supply Chain Security

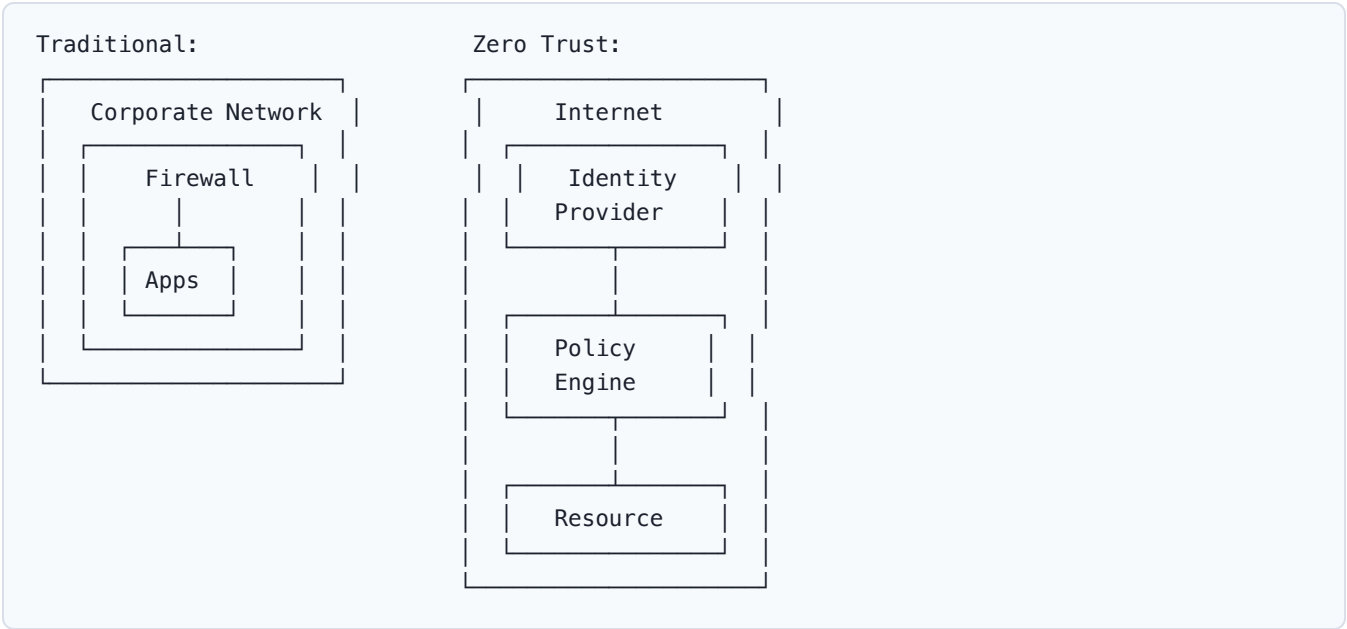
Zero Trust

Zero Trust en Desarrollo

Principles

Principle	Descripción
Never Trust, Always Verify	Verificar cada request explícitamente
Least Privilege	Acceso mínimo necesario
Assume Breach	Asumir que la red está comprometida
Verify Explicitly	Autenticar y autorizar cada request

Arquitectura Zero Trust



Componentes

1. Identity Provider (IdP)

```
# OIDC Configuration
identity:
  provider: keycloak
  protocols:
    - oidc
    - saml
```

```
mfa:
  required: true
  methods:
    - totp
    - webauthn
  session:
    max_duration: 8h
    idle_timeout: 30m
```

2. Policy Engine

```
# OPA Policy
package authz

default allow = false

# Allow if authenticated and authorized
allow {
  input.method == "GET"
  input.path == ["api", "resources"]
  token.valid
  token.payload.role == "reader"
}

allow {
  input.method == "POST"
  input.path == ["api", "resources"]
  token.valid
  token.payload.role == "writer"
}

# Verify MFA
allow {
  token.payload.mfa_verified == true
}
```

3. Service Mesh (mTLS)

```
# Istio: AuthorizationPolicy
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-frontent-to-backend
spec:
  selector:
    matchLabels:
      app: backend
  action: ALLOW
```

```
rules:
  - from:
      - source:
          principals: ["cluster.local/ns/default/sa/frontend"]
    to:
      - operation:
          methods: ["GET", "POST"]
          paths: ["/api/*"]
```

mTLS Implementation

```
# Istio PeerAuthentication
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system
spec:
  mtls:
    mode: STRICT # Force mTLS for all services
```

Certificate Management

```
# Cert Manager for automatic certificates
kubectl apply -f - <<EOF
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-cert
  namespace: default
spec:
  secretName: app-tls
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - app.example.com
EOF
```

Implementation Phases

Phase 1: Identity

```
# Implement MFA
mfa:
  methods:
    - totp: Google Authenticator
```

- webauthn: Hardware keys
- enforcement:
- admin_interfaces: required
 - api_access: required
 - standard_users: optional

Phase 2: Device Trust

```
# Device compliance check
device_trust:
  requirements:
    - os_version: latest
    - disk_encryption: enabled
    - antivirus: active
    - firewall: enabled
  non_compliant_action: block
```

Phase 3: Network Microsegmentation

```
# Network policies (Kubernetes)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

Phase 4: Application Access

```
# Proxy-based access
zero_trust_proxy:
  provider: cloudflare-access
  policies:
    - name: admin-access
```

```
include:
  - email: admin@example.com
  - group: admin-group
require:
  - mfa
  - device_posture: compliant
```

Zero Trust for Development

Component	Traditional	Zero Trust
Code Access	VPN to repo	Identity-based access
Secrets	Environment variables	Vault with dynamic secrets
CI/CD	Shared runners	Isolated, authenticated
Deployments	SSH + keys	Signed commits + OIDC
Monitoring	Centralized logs	Distributed tracing

Dynamic Secrets

```
# Vault: Dynamic database credentials
import hvac

client = hvac.Client(url='https://vault.example.com')

# Generate dynamic credentials
secret = client.secrets.database.generate_credentials(
    name='my-role'
)

# Secret expires automatically
username = secret['username'] # v-user-abc123
password = secret['password'] # A1b2C3d4E5
```

Ver también

- DevSecOps
- Supply Chain Security
- Compliance
- OWASP Top 10
- Pipelines

Supply Chain Security

Seguridad de Cadena de Suministro

Amenazas Comunes

Amenaza	Descripción	Ejemplo
Dependency Confusion	Paquetes maliciosos en repositorios públicos	event-stream incident
Typosquatting	Paquetes con nombres similares	lodash-es vs lodash
Compromised Maintainer	Cuenta de mantenedor comprometida	ua-parser-js
Build Compromise	Sistema de build comprometido	SolarWinds
Source Code Tampering	Modificación del código fuente	codecov breach

SBOM (Software Bill of Materials)

¿Qué es un SBOM?

Inventario completo de componentes de software, dependencias y relaciones.

```
{
  "spdxVersion": "SPDX-2.3",
  "dataLicense": "CC0-1.0",
  "name": "my-application",
  "packages": [
    {
      "name": "lodash",
      "version": "4.17.21",
      "supplier": "Organization: lodash",
      "downloadLocation": "https://registry.npmjs.org/lodash",
      "checksums": [
        {
          "algorithm": "SHA256",
          "value": "..."
        }
      ]
    }
  ]
}
```

Herramientas SBOM

Herramienta	Lenguaje	Formato
Syft	Multi	SPDX, CycloneDX
CycloneDX	Multi	CycloneDX
FOSSA	Multi	SPDX
Tern	Containers	SPDX

Generar SBOM

```
# Syft: Generar SBOM
syft scan dir:. -o spdx > sbom.spdx.json
syft scan dir:. -o cyclonedx-json > sbom.cdx.json

# Docker image SBOM
syft scan docker:myapp:latest -o spdx > sbom.spdx.json

# Escanear SBOM
grype sbom:sbom.spdx.json
```

SLSA (Supply-chain Levels for Software Artifacts)

Niveles

Level	Requisito	Protección
L1	Build process documented	Provenance
L2	Hosted build, signed provenance	Tampering
L3	Hardened builds, non-falsifiable	Insider threat
L4	Reproducible builds, hermetic	Compromise

Provenance

```
{
  "predicateType": "https://slsa.dev/provenance/v0.2",
  "predicate": {
    "builder": {
      "id": "https://github.com/actions/runner"
    },
    "buildType": "https://github.com/actions/workflow",
    "invocation": {
      "configSource": {
        "uri": "git+https://github.com/org/repo@refs/heads/main",
        "digest": {
```

```

        "sha1": "abc123..."
      }
    }
  },
  "metadata": {
    "buildInvocationId": "https://github.com/org/repo/actions/runs/123",
    "buildStartedOn": "2026-01-13T00:00:00Z",
    "completeness": {
      "parameters": true,
      "environment": true,
      "materials": true
    },
    "reproducible": true
  }
}

```

Sigstore / Cosign

Firmar Contenedores

```

# Cosign: Firma de contenedores
cosign sign --key cosign.key myregistry/myapp:latest

# Verificar firma
cosign verify --key cosign.pub myregistry/myapp:latest

# Firma con keyless (OIDC)
cosign sign --identity-token $OIDC_TOKEN myregistry/myapp:latest

```

Software Signing Pipeline

```

# GitHub Actions: Sign artifacts
sign-artifacts:
  runs-on: ubuntu-latest
  permissions:
    id-token: write
    contents: write
  steps:
    - uses: actions/checkout@v4
    - uses: sigstore/cosign-installer@v3

    - name: Build and push
      run: |
        docker build -t myregistry/app:${{ github.sha }} .
        docker push myregistry/app:${{ github.sha }}

    - name: Sign container

```

```

    run: cosign sign myregistry/app:${{ github.sha }}

- name: Attach SBOM
  run: |
    syft scan docker:myregistry/app:${{ github.sha }} \
      -o spdx > sbom.spdx.json
    cosign attach sbom \
      --sbom sbom.spdx.json \
      myregistry/app:${{ github.sha }}

```

Dependabot / Renovate

Dependabot Configuration

```

# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 10
    reviewers:
      - "security-team"
    labels:
      - "dependencies"
      - "security"

```

Renovate Configuration

```

{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": [
    "config:recommended",
    ":security"
  ],
  "vulnerabilityAlerts": {
    "enabled": true,
    "labels": ["security"]
  },
  "packageRules": [
    {
      "matchUpdateTypes": ["patch"],
      "automerger": true
    }
  ]
}

```

```
]
}
```

Container Security

Image Signing Verification

```
# Kyverno: Enforce image signatures
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-image-signatures
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-image-signature
      match:
        any:
          - resources:
              kinds:
                - Pod
      verifyImages:
        - imageReferences:
            - "myregistry/*"
          attestors:
            - entries:
                - keys:
                    publicKey: |-
                      -----BEGIN PUBLIC KEY-----
                      ...
                      -----END PUBLIC KEY-----
```

Dependency Review

```
# GitHub: Dependency Review
name: Dependency Review
on: pull_request
jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/dependency-review-action@v3
    with:
      fail-on-severity: high
      deny-licenses: GPL-3.0, AGPL-3.0
```

Ver también

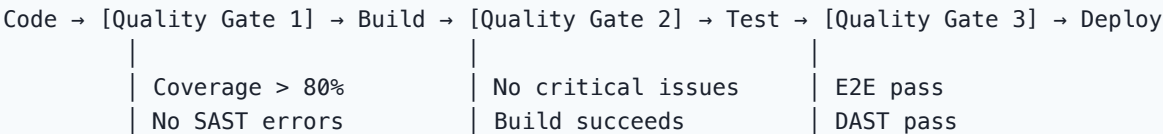
- [DevSecOps](#)
- [Zero Trust](#)
- [Security Testing](#)
- [OWASP Top 10](#)
- [Compliance](#)

Quality Gates

Quality Gates en CI/CD

¿Qué es un Quality Gate?

Punto de verificación en el pipeline que debe cumplirse antes de avanzar a la siguiente etapa.



Tipos de Quality Gates

Gate	Fase	Criterios
Commit	Pre-commit	Lint, format, secrets
PR	Code review	Tests, coverage, SAST
Build	CI	Compilation, unit tests
Deploy	CD	Integration tests, DAST
Production	Release	Performance, security

Criterios Comunes

Code Quality

```
quality_gates:
  coverage:
    metric: "line_coverage"
    threshold: 80
    type: "greater_than"

  duplications:
    metric: "duplicated_lines"
    threshold: 3
    type: "less_than"

  complexity:
    metric: "cyclomatic_complexity"
    threshold: 10
    type: "less_than"
```

```
violations:
  metric: "new_violations"
  threshold: 0
  type: "equal"
```

Security

```
security_gates:
  sast:
    tool: semgrep
    fail_on:
      - severity: "ERROR"
      - confidence: "HIGH"

  sca:
    tool: snyk
    fail_on:
      - severity: "critical"
      - severity: "high"

  secrets:
    tool: gitleaks
    fail_on: any_detection
```

Performance

```
performance_gates:
  response_time:
    metric: "p95_latency"
    threshold: 500 # ms
    type: "less_than"

  error_rate:
    metric: "http_errors"
    threshold: 1 # percent
    type: "less_than"

  throughput:
    metric: "requests_per_second"
    threshold: 100
    type: "greater_than"
```

Implementación con SonarQube

Quality Gate Configuration


```
# sonarqube-quality-gate.yml
name: "Custom Quality Gate"
conditions:
  - metric: new_coverage
    operator: LESS_THAN
    value: 80
    period: NEW_CODE

  - metric: new_duplicated_lines_density
    operator: GREATER_THAN
    value: 3
    period: NEW_CODE

  - metric: new_bugs
    operator: GREATER_THAN
    value: 0
    period: NEW_CODE

  - metric: new_vulnerabilities
    operator: GREATER_THAN
    value: 0
    period: NEW_CODE
```

CI Integration

```
# GitHub Actions: SonarQube
sonarqube:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0
    - uses: sonarqube-community/sonarqube-scan-action@master
      env:
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
    - uses: sonarqube-community/sonarqube-quality-gate-action@master
      timeout-minutes: 5
      env:
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

GitHub Status Checks

```
# branch protection rules
branches:
  main:
    required_status_checks:
      strict: true
```

```
contexts:
  - "coverage/patch"
  - "security/sast"
  - "security/sca"
  - "tests/unit"
  - "tests/integration"
required_pull_request_reviews:
  required_approving_review_count: 1
dismiss_stale_reviews: true
```

Custom Gate Scripts

```
#!/bin/bash
# quality-gate.sh

set -e

echo "=== Quality Gate Checks ==="

# Check coverage
COVERAGE=$(cat coverage/coverage-summary.json | jq '.total.lines.pct')
if (( $(echo "$COVERAGE < 80" | bc -l) )); then
  echo "FAIL: Coverage $COVERAGE% < 80%"
  exit 1
fi
echo "PASS: Coverage $COVERAGE%"

# Check vulnerabilities
VULNS=$( snyk test --severity-threshold=high --json | jq '.vulnerabilities | length' )
if [ "$VULNS" -gt 0 ]; then
  echo "FAIL: $VULNS high/critical vulnerabilities"
  exit 1
fi
echo "PASS: No high/critical vulnerabilities"

# Check code smells
ISSUES=$(sonar-scanner -q -Dsonar.qualitygate.wait=true | jq '.status')
if [ "$ISSUES" != "OK" ]; then
  echo "FAIL: Quality gate not met"
  exit 1
fi
echo "PASS: Quality gate met"

echo "=== All checks passed ==="
```

Metrics Dashboard

Metric	Current	Target	Status
Code Coverage	82%	80%	
Duplications	2.5%	3%	
Bugs	0	0	
Vulnerabilities	1	0	⚠️
Security Hotspots	3	0	⚠️
Technical Debt	2.5 days	5 days	

Best Practices

Practice	Descripción
Fail Fast	Gate más rápido primero
Automate	Todo automatizado, no manual
New Code Focus	Evaluar solo código nuevo
Progressive	Gates más estrictos en producción
Block/Warn	Diferenciar errores de warnings

Ver también

- QA Moderno
- Pirámide de Testing
- Testing Automatizado
- Security Testing
- Prevención de Defectos
- Pipelines CI/CD

Defect Prevention

Prevención de Defectos vs Detección

Filosofía

Enfoque	Costo	Eficiencia	Momento
Prevención	Bajo	Alta	Antes de 编码
Detección	Medio	Media	Durante testing
Corrección	Alto	Baja	En producción

"Es 100x más barato prevenir un defecto que corregirlo en producción."

Métodos de Prevención

Poka-Yoke (Error-Proofing)

Mecanismos que previenen errores humanos antes de que ocurran.

Tipos

Tipo	Función	Ejemplo
Preventivo	Impide el error	Conector USB asimétrico
Detectivo	Detecta el error	Alarma de puerta abierta
Correctivo	Corrige el error	Auto-correct en typing

Ejemplos en Software

```
// Type safety - Preventivo
type UserRole = 'admin' | 'user' | 'viewer';

function assignRole(role: UserRole): void {
  // TypeScript previene roles inválidos en compile time
  user.role = role;
}

// assignRole('superadmin'); // Error de compilación

// Input validation - Preventivo
function processAge(age: unknown): number {
  if (typeof age !== 'number') {
```

```
        throw new TypeError('Age must be a number');
    }
    if (age < 0 || age > 150) {
        throw new RangeError('Age must be between 0 and 150');
    }
    return age;
}

// Null safety - Detective
function getUsername(user: User | null): string {
    return user?.name ?? 'Unknown'; // Null check with fallback
}
```

Poka-Yoke en Frontend

```
// React: Form validation
const LoginForm = () => {
    const schema = z.object({
        email: z.string().email(),
        password: z.string().min(8),
    });

    return (
        <Form onSubmit={handleSubmit}>
            <Input
                type="email"
                required
                pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$"
                title="Please enter a valid email"
            />
            <Input
                type="password"
                required
                minLength={8}
                title="Password must be at least 8 characters"
            />
        </Form>
    );
};
```

FMEA (Failure Mode and Effects Analysis)

Step	Descripción
1. Identify	Identificar posibles fallos
2. Analyze	Evaluar severidad, probabilidad, detectabilidad
3. Prioritize	Calcular RPN (Risk Priority Number)

Step	Descripción
4. Action	Implementar acciones preventivas
5. Verify	Verificar efectividad

FMEA Template

Component	Failure Mode	Effect	Severity	Cause	Occurrence	Detection	RPN	Action
Auth module	Token expired	User locked out	8	No refresh	5	3	120	Auto-refresh
Database	Connection timeout	App crash	9	High load	3	4	108	Connection pooling
API Gateway	Rate limit hit	429 errors	5	Traffic spike	6	2	60	Backoff retry

FMEA Calculation

RPN = Severity × Occurrence × Detection

Range: 1–1000

- High Risk: RPN > 200 → Immediate action
Medium Risk: RPN 100–200 → Plan action
Low Risk: RPN < 100 → Monitor

Design Reviews

```
# Code Review Checklist
design_review:
  architecture:
    - separation_of_concerns
    - single_responsibility
    - dependency_inversion
  error_handling:
    - graceful_degradation
    - retry_logic
    - circuit_breaker
  security:
    - input_validation
    - authentication
    - authorization
  performance:
    - caching_strategy
```

- lazy_loading
- pagination

Static Analysis

```
# ESLint Rules for Prevention
rules:
  # Prevent null/undefined errors
  - "@typescript-eslint/no-unnecessary-condition"
  - "no-optional-chaining"

  # Prevent type errors
  - "@typescript-eslint/strict-boolean-expressions"
  - "@typescript-eslint/no-unsafe-assignment"

  # Prevent logic errors
  - "no-duplicate-case"
  - "no-unreachable"
  - "eqeqeq"

  # Prevent security issues
  - "no-eval"
  - "no-implied-eval"
  - "@typescript-eslint/no-non-null-assertion"
```

Defect Prevention Pipeline

```
# Prevention-focused pipeline
stages:
  - name: design
    checks:
      - architecture_review
      - threat_modeling
      - fMEA_analysis

  - name: development
    checks:
      - type_safety
      - input_validation
      - error_handling

  - name: code_review
    checks:
      - peer_review
      - security_review
      - performance_review

  - name: quality_gate
```

- checks:
- unit_test_coverage
 - integration_test_coverage
 - security_scan

Cost-Benefit Analysis

Defect Stage	Cost to Fix	Relative Cost
Requirements	\$1	1x
Design	\$5	5x
Code	\$10	10x
Unit Test	\$20	20x
Integration Test	\$50	50x
System Test	\$100	100x
Production	\$1,000+	1,000x

Root Cause Analysis

5 Whys

Problem: Production outage

1. Why? Server crashed → Out of memory

2. Why? Memory leak → Event listener not cleaned

3. Why? Component re-renders → State update triggers

4. Why? No cleanup in useEffect → Missing dependency

5. Why? No code review caught it → No checklist for React cleanup

Action: Add cleanup checklist to code review

Fishbone Diagram Categories

Category	Examples
People	Training, fatigue, knowledge gaps
Process	Missing reviews, unclear requirements
Tools	Outdated IDE, missing linters
Environment	Network issues, load

Ver también

- QA Moderno

- Pirâmide de Testing
- Quality Gates
- Security Testing
- OWASP Top 10

README

Calidad y Seguridad

Visión general

La calidad y la seguridad son pilares fundamentales del desarrollo de software moderno. Esta sección cubre desde las prácticas de QA hasta la seguridad integral del ciclo de vida del desarrollo.

Contenido de la sección

Archivo	Tema	Dificultad	
[[01-qa-moderno	QA Moderno]]	Más allá del testing manual	Principiante
[[02-testing-piramide	Pirámide de Testing]]	Estrategia de testing	Intermedio
[[03-automated-testing	Testing Automatizado]]	Herramientas y frameworks	Intermedio
[[04-performance-testing	Performance Testing]]	Load testing y benchmarks	Avanzado
[[05-security-testing	Security Testing]]	SAST, DAST, SCA	Avanzado
[[06-owasp-top10	OWASP Top 10]]	Vulnerabilidades críticas	Intermedio
[[07-devsecops	DevSecOps]]	Seguridad integrada	Avanzado
[[08-compliance-regulaciones	Compliance]]	GDPR, SOC2, HIPAA	Avanzado
[[09-zero-trust	Zero Trust]]	Modelo de seguridad	Avanzado
[[10-supply-chain-security	Supply Chain Security]]	SBOM, SLSA	Avanzado
[[11-quality-gates	Quality Gates]]	Control de calidad en CI/CD	Intermedio
[[12-defect-prevention	Prevención de Defectos]]	Poka-Yoke, FMEA	Intermedio

Conceptos clave

Shift-Left Security

Integrar la seguridad desde las primeras fases del desarrollo, no como una fase final. Ver DevSecOps.

Quality Engineering

Evolution del QA tradicional hacia la ingeniería de calidad como disciplina integral. Ver QA Moderno.

Defense in Depth

Múltiples capas de seguridad que se complementan entre sí. Ver Zero Trust.

Relación con otras secciones

- **Fundamentos:** ¿Qué es DevOps?
- **Procesos:** CI/CD
- **Prácticas:** Code Review
- **Herramientas:** Git
- **Métricas:** DORA Metrics
- **DevOps:** Pipelines